

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Современные платформы программирования

Лабораторная работа №1

ТЕМА
«Приложение Pastebin SSR»

Студент: гр. 351001 Матюшенко А.Д.

Проверила: Петровская В. В.

Минск 2026

ВВЕДЕНИЕ

«Pastebin SSR» – серверное веб-приложение для публикации, хранения и совместного доступа к фрагментам исходного кода, конфигурационным файлам и текстовым логам с серверным рендерингом (Server-Side Rendering, SSR). Функционал разработанного программного обеспечения включает в себя:

- главный экран каталога сниппетов с выводом списка доступных записей, поддержкой полнотекстового поиска, фильтрации по языкам программирования и статусу активности (активные / истёкшие), а также отображением количества просмотров, даты публикации, срока жизни и индикатора прикреплённых файлов;
- экран добавления нового сниппета с интерактивной формой, включающей выбор языка подсветки синтаксиса, редактор кода с моноширинным шрифтом, загрузку прикреплённых файлов (до 10 МБ), гибкую настройку времени хранения (от 10 минут до бессрочного либо точной даты) и опцию приватности (Unlisted);
- экран детального просмотра сниппета с серверной подсветкой синтаксиса в теме оформления GitHub Dark, отображением счётчика строк и метаданных, блоком скачивания прикреплённого файла, кнопками быстрого копирования URL-ссылки, копирования кода, перехода к Raw-просмотру и удаления записи;
- эндпоинт отдачи необработанного текстового содержимого (Raw View) с заголовком Content-Type: text/plain;
- фоновую службу автоматической очистки базы данных и файлового хранилища от устаревших записей с истёкшим сроком жизни.

Для разработки используется среда выполнения NodeJS и строго типизированный язык программирования TypeScript, что обеспечивает высокую надёжность кодовой базы, строгий контроль типов моделей данных и отсутствие типичных ошибок динамической типизации. В рамках выбранного технологического стека применены следующие библиотеки:

- **express** – быстрый и минималистичный веб-фреймворк для организации HTTP-маршрутизации и middleware;
- **ejs** – гибкий шаблонизатор для генерации готовой HTML-разметки на стороне сервера с разделением на компоненты (partials);
- **better-sqlite3** – высокопроизводительная встраиваемая реляционная СУБД SQLite для надёжного локального хранения метаданных сниппетов без внешних зависимостей;

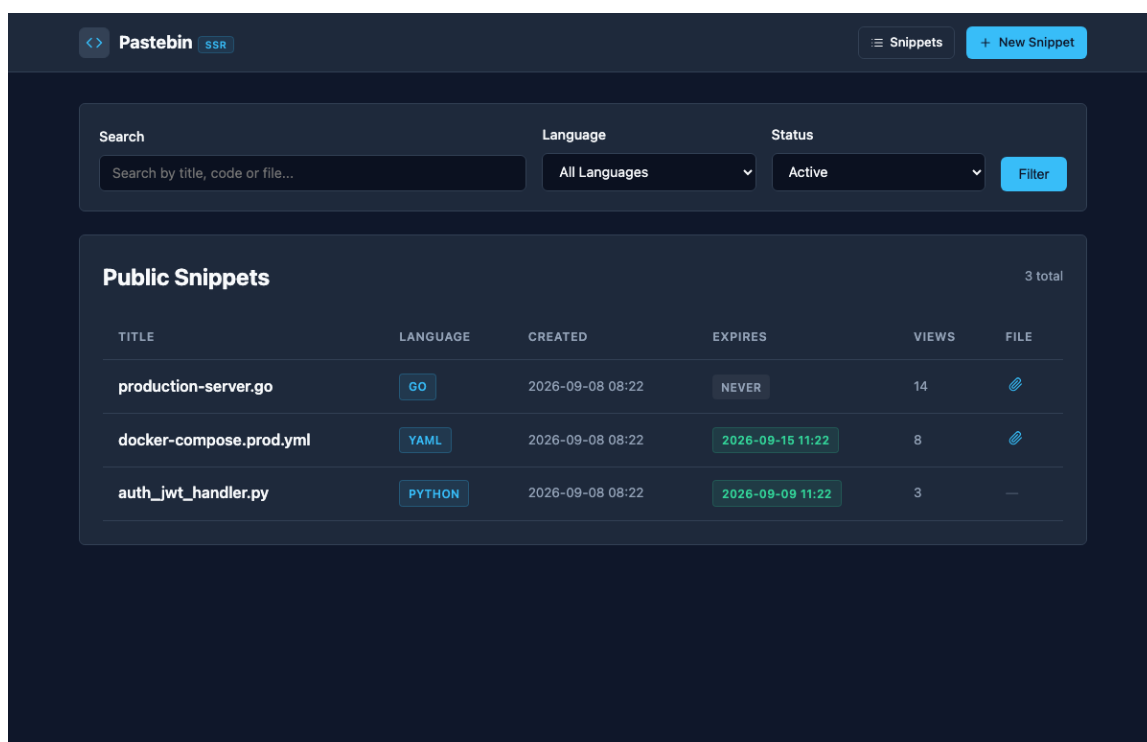
- **multer** – специализированный middleware для обработки потоковых многокомпонентных запросов (multipart/form-data) и сохранения файлов на диск;
- **highlight.js** – библиотека серверной подсветки синтаксиса более чем 15 языков программирования (Go, TypeScript, Python, SQL, YAML и др.) до передачи HTML клиенту;
- **dayjs** – компактная утилита для манипуляций с датами, форматирования меток времени и вычисления сроков жизни;
- **nanoid** – генератор компактных и безопасных уникальных строковых идентификаторов (slug) для адресации сниппетов.

Архитектура приложения спроектирована по модульному принципу с чётким разделением ответственности: слой данных Data Layer (база данных SQLite и репозиторий snippetRepository), сервисный слой Service Layer (бизнес-логика, подсветка синтаксиса, расчёт сроков хранения в snippetService), транспортный слой Transport Layer (обработчики маршрутов snippetRoutes, middleware загрузки multer) и слой представления Presentation Layer (серверные шаблоны EJS и CSS-стили).

1 СКРИНШОТЫ РАБОТЫ ПРИЛОЖЕНИЯ С ОПИСАНИЕМ ФУНКЦИЙ

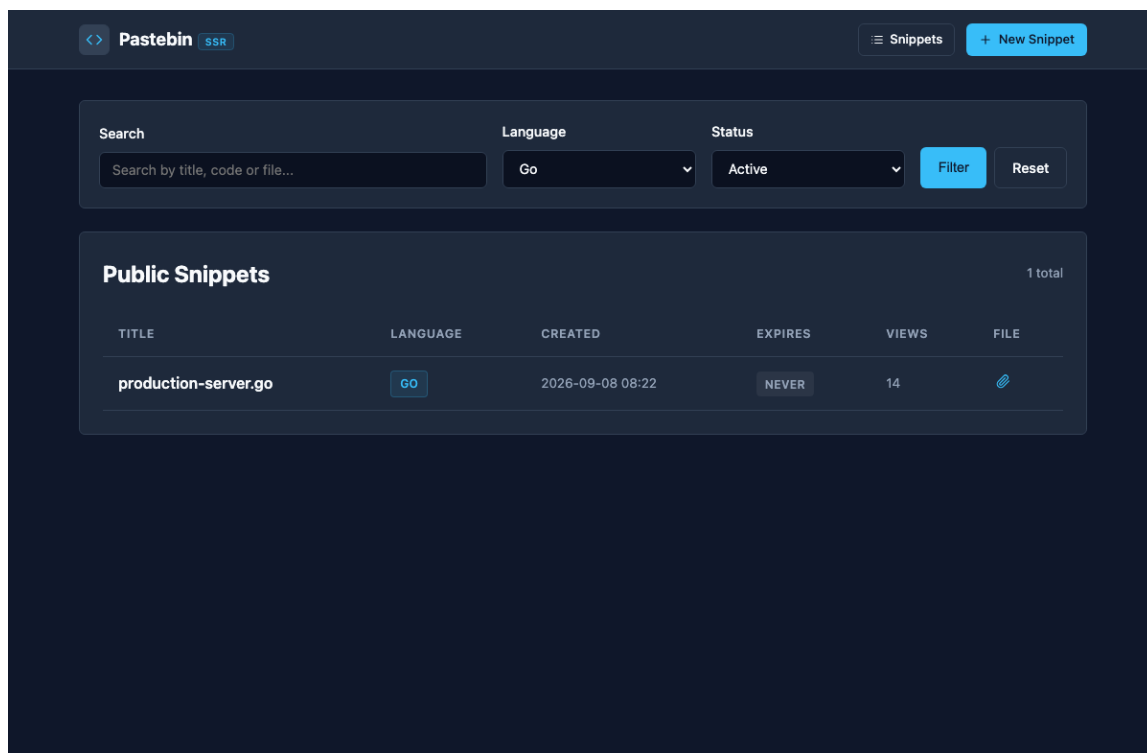
Главный экран каталога сниппетов

Главный экран приложения содержит верхнюю навигационную панель со стилизованным логотипом и кнопками перехода, панель фильтрации и поиска, а также таблицу опубликованных публичных записей. В таблице для каждого сниппета отображаются название, бейдж используемого языка программирования, дата и время создания, оставшийся срок хранения, количество уникальных просмотров и статус наличия прикрепленного файла. Все фильтры работают через стандартную форму отправки данных методом GET с query-параметрами.



Поиск и фильтрация сниппетов по критериям

При заполнении полей поискового запроса, выборе конкретного языка программирования или статуса (Active / Expired / All) сервер формирует параметризованный SQL-запрос к базе данных SQLite и отдаёт клиенту готовый HTML с отфильтрованным набором данных. При активном фильтре интерфейс выводит общее количество найденных записей и предоставляет кнопку «Reset» для быстрого сброса критериев поиска.



Экран создания нового сниппета и загрузки файлов

Экран добавления представляет собой форму отправки данных методом POST с кодировкой multipart/form-data. Форма включает поля для названия сниппета, выпадающий список выбора языка синтаксиса, редактор кода с моноширинным шрифтом, поле выбора файла для прикрепления (до 10 МБ), блок выбора срока хранения (пресеты от 10 минут до 1 месяца, бессрочно либо выбор точной даты через datetime-local), а также переключатель режима приватности Unlisted. Если поле с кодом не заполнено, сервер автоматически считывает текст из прикрепленного текстового файла.

The screenshot shows the 'Create New Snippet' interface of the Pastebin SSR application. The header includes the 'Pastebin SSR' logo and a 'Snippets' menu. The main form is titled 'Create New Snippet' and contains the following elements:

- Title / Description:** A text input field with the placeholder 'e.g. main.go or docker-compose.yml'.
- Syntax Language:** A dropdown menu currently set to 'Plain Text'.
- Code or Text Content:** A large text area with the placeholder 'Paste your code, configuration or log text here...'.
- Attach File:** A section with a 'Choose File' button and the text 'No file chosen'. Below it, a note states: 'Maximum file size: 10 MB. Can be downloaded from the snippet page.'
- Expiration:** A dropdown menu currently set to 'Never'.
- Unlisted:** A checkbox labeled 'Unlisted (Hidden from public snippet list, accessible only via direct link)'.

Экран детального просмотра snippets с подсветкой синтаксиса

При переходе по уникальной ссылке snippets сервер считывает запись из SQLite, инкрементирует счётчик просмотров и выполняет парсинг синтаксиса через highlight.js, оборачивая лексемы кода в CSS-классы темы GitHub Dark. На странице отображаются бейдж языка, бейдж срока жизни, дата создания, количество строк и просмотров. В правой верхней части расположены кнопки «Share Link» (копирование ссылки в буфер обмена), «Raw Text» (переход к чистому тексту) и «Delete» (удаление записи). При наличии прикрепленного файла выводится карточка вложения с кнопкой скачивания через эндпоинт /snippets/:id/download.

The screenshot shows the Pastebin SSR web interface. At the top, there's a navigation bar with the Pastebin logo, a 'Snippets' menu, and a 'New Snippet' button. The main content area displays a snippet titled 'production-server.go'. Below the title, there are buttons for 'Share Link', 'Raw Text', and 'Delete'. A status bar indicates 'GO', 'NEVER EXPIRES', 'Created: 2026-09-08 08:22', 'Views: 15', and 'Lines: 36'. Below this, there's a section for the file 'main.go' (0.8 KB, text/x-go) with a 'Download Attachment' button. The main part of the interface shows the Go code in a dark-themed editor with a 'Copy Code' button in the top right corner.

```
package main

import (
    "context"
    "fmt"
    "log"
    "net/http"
    "time"
)

func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    })
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Welcome to Pastebin SSR API!")
    })
}
```

Просмотр исходного содержимого в формате Raw Text

При обращении к эндпоинту `/snippets/:id/raw` сервер отдаёт клиенту тело скрипта в чистом виде без HTML-разметки и стилей с HTTP-заголовком `Content-Type: text/plain; charset=utf-8`. Данная функция позволяет скачивать исходные скрипты и конфиги напрямую из терминала с помощью утилит `curl` и `wget`, либо использовать их во внешних автоматизированных скриптах.

```

package main

import (
    "context"
    "fmt"
    "log"
    "net/http"
    "time"
)

func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    })
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Welcome to Pastebin SSR API!")
    })

    server := &http.Server{
        Addr:    ":8080",
        Handler: loggingMiddleware(mux),
        ReadTimeout: 5 * time.Second,
        WriteTimeout: 10 * time.Second,
    }

    log.Println("Server listening on http://localhost:8080")
    if err := server.ListenAndServe(); err != nil {
        log.Fatal(err)
    }
}

```

Отображение состояния истёкшего срока хранения

В случае если установленный срок хранения сниппета подошёл к концу, система отображает статус «Expired» и специальный предупреждающий баннер на странице просмотра. Такие записи автоматически скрываются из каталога активных сниппетов на главной странице и безвозвратно удаляются из базы данных и файловой системы uploads/ фоновым планировщиком очистки каждые 30 минут.

The screenshot shows the Pastebin SSR web interface. At the top, there's a navigation bar with a code icon, the text 'Pastebin SSR', a 'Snippets' menu, and a '+ New Snippet' button. The main content area displays a snippet titled 'v1_initial_schema_migration.sql'. To the right of the title are buttons for 'Share Link', 'Raw Text', and 'Delete'. Below the title, a status bar indicates 'SQL', 'EXPIRED (2026-09-08 06:22)', and metadata: 'Created: 2026-09-08 08:22 • Views: 1 • Lines: 10'. A yellow warning box states: 'This snippet has expired and will be automatically purged during the next cleanup cycle.' Below this, the snippet's content is shown in a dark-themed editor with a 'Copy Code' button. The code is SQL for creating a 'users' table and an index.

```

SQL
Copy Code

-- Migration: V1 Initial Schema
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

```


ВЫВОД

В ходе выполнения лабораторной работы №1 было разработано веб-приложение для хранения, каталогизации и обмена фрагментами кода с серверным рендерингом (SSR) на технологическом стеке NodeJS, TypeScript, Express, EJS и SQLite. Были освоены и практически закреплены следующие ключевые концепции:

- организация серверного рендеринга динамических веб-страниц с использованием шаблонизатора EJS и повторно используемых компонентов-фрагментов (partials);
- обработка пользовательских запросов и передача данных исключительно посредством стандартных HTML-форм (методы GET для фильтрации и POST для модификации данных);
- приём, валидация и безопасное сохранение бинарных и текстовых файлов произвольного формата на стороне сервера с помощью потокового middleware Multer;
- встраивание и работа с реляционной локальной базой данных SQLite через библиотеку better-sqlite3 с использованием транзакций и подготовленных SQL-выражений (Prepared Statements);
- реализация серверной подсветки синтаксиса через highlight.js, механизма управления жизненным циклом и сроками хранения записей, а также фоновой очистки устаревших данных.

Исходный код проекта структурирован, строго типизирован, протестирован и размещён в удалённом репозитории GitHub:

<https://github.com/fluffy11lol/spp>